

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Lab Manual 9

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 Single Perceptron	3
1.1 Types of Perceptrons	3
1.2 Perceptron Function	3
1.3 Inputs of a Perceptron	4
1.4 Activation Functions of Perceptron	5
1.5 Perceptron at a glance	5
1.6 Perceptron Training Algorithms	5
1.7 Logic Gates	6
2 Introduction to tensorflow and pytorch	7
2.1 What is TensorFlow?	7
2.2 What is PyTorch?	7
3 Single-Layer Perceptron using Pytorch	8
3.1 Import necessary libraries	8
3.2 Create and split synthetic dataset	8
3.3 Standardize the Dataset	8
3.4 Convert Data to PyTorch Tensors	9
3.5 Building a neural network	11
3.6 Define Loss Function and Optimizer	15
3.7 Train the Model	17
3.8 Model Evaluation	18

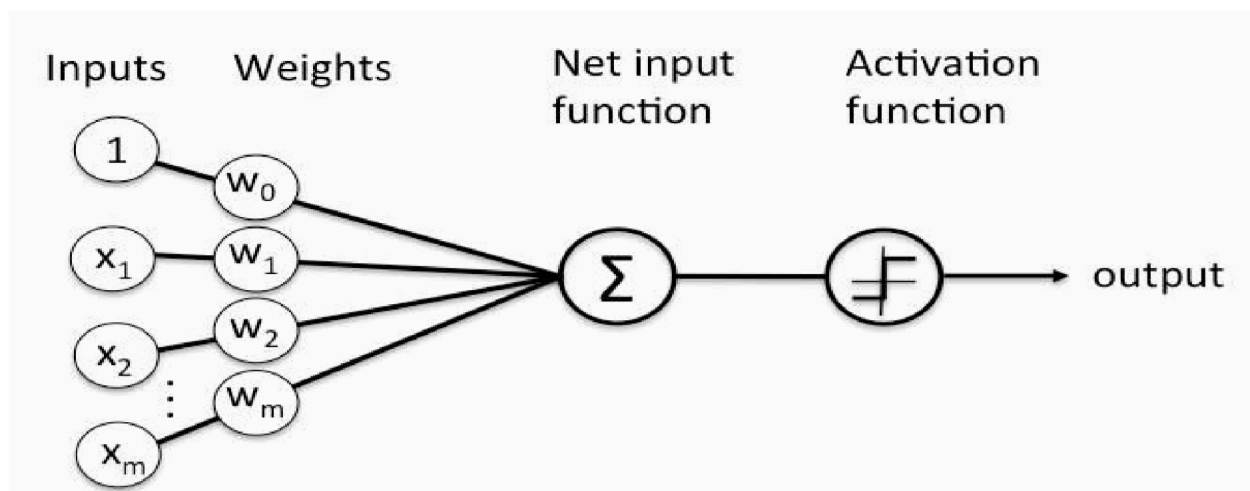
Objectives

After performing this lab, students shall be able to:

- ✓ Understand single layer perceptron
- ✓ Implement it using tensorflow

1 Single Perceptron

A perceptron is a neural network unit (an artificial neuron) that does certain computations to detect features or business intelligence in the input data. A Perceptron is an algorithm for supervised learning of binary classifiers. This algorithm enables neurons to learn and process elements in the training set one at a time.



1.1 Types of Perceptrons

- 1) **Single layer Perceptrons** can learn only linearly separable patterns.
- 2) **Multilayer Perceptrons** or feedforward neural networks with two or more layers have the greater processing power.

1.2 Perceptron Function

Perceptron is a function that maps its input “x,” which is multiplied with the learned weight coefficient; an output value “f(x)” is generated.

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

In the equation given above:

“w” = vector of real-valued weights

“b” = bias (an element that adjusts the boundary away from origin without any dependence on the input value)

“x” = vector of input x values

$$\sum_{i=1}^m w_i x_i$$

“m” = number of inputs to the Perceptron

The output can be represented as “1” or “0.” It can also be represented as “1” or “-1” depending on which activation function is used.

1.3 Inputs of a Perceptron

A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result. The image below shows a Perceptron with a Boolean output.

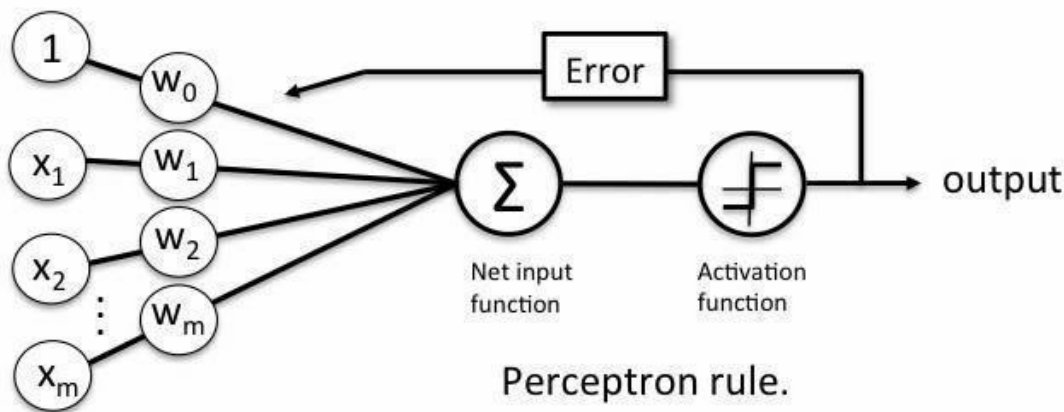


Figure 2 Inputs in a Perceptron Learning Process

A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False. The summation function “ Σ ” multiplies all inputs of “x” by weights “w” and then adds them up as follows:

$$w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

1.4 Activation Functions of Perceptron

The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.

For example:

If $\sum w_i x_i > 0 \Rightarrow$ then final output "o" = 1 (issue bank loan)
 Else, final output "o" = -1 (deny bank loan)

Step function gets triggered above a certain value of the neuron output; else it outputs zero.

1.5 Perceptron at a glance

- Weights are multiplied with the input features and decision is made if the neuron is fired or not.
- Activation function applies a step rule to check if the output of the weighting function is greater than zero.
- Linear decision boundary is drawn enabling the distinction between the two linearly separable classes +1 and -1.
- If the sum of the input signals exceeds a certain threshold, it outputs a signal; otherwise, there is no output.
- Types of activation functions include the sign, step, and sigmoid functions.

1.6 Perceptron Training Algorithms

In this course, we have covered training algorithms namely, perceptron learning.

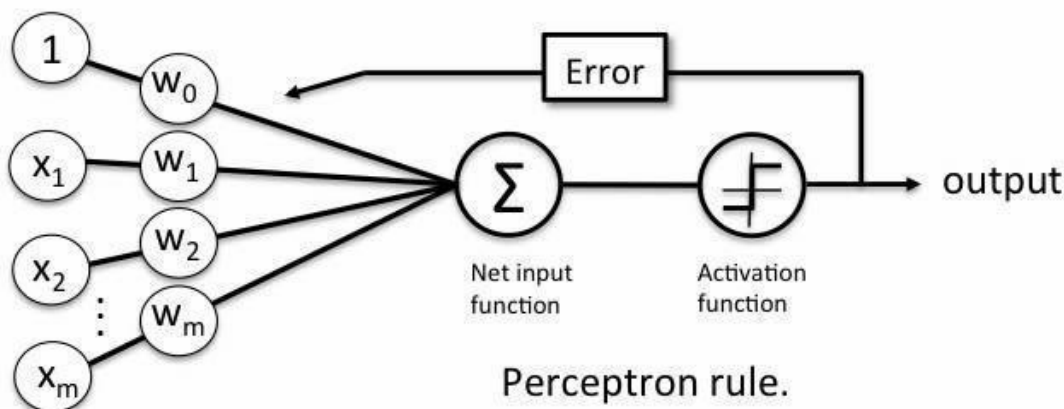


Figure 4 Learning Process

The perceptron receives multiple input signals and if the sum of the input signals exceeds a certain threshold, it either outputs a signal or does not return an output.

In this lab session, we only focus on the perceptron learning rule. This rule requires the data to be linearly separable. The weight update rule is given by:

$$w_i = w_i + \Delta w_i$$

where

$$\Delta w_i = \eta \cdot (y - \hat{y}) \cdot x_i$$

and

$$b = b + \eta \cdot (y - \hat{y})$$

Where:

- y is the actual output,
- $\hat{y}$ is the perceptron's predicted output,
- η is the learning rate (a small constant that controls how much weights change).

1.7 Logic Gates

Logic gates are the building blocks of a digital system, especially neural networks. In short, they are the electronic circuits that help in addition, choice, negation, and combination to form complex circuits.

Using the logic gates, Neural Networks can learn on their own without you having to manually code the logic. Most logic gates have two inputs and one output.

Each terminal has one of the two binary conditions, low (0) or high (1), represented by different voltage levels. The logic state of a terminal changes based on how the circuit processes data. Based on this logic, logic gates can be categorized into six types:

- 1) AND
- 2) NAND
- 3) OR
- 4) NOR
- 5) NOT
- 6) XOR

2 Introduction to tensorflow and pytorch

Modern Artificial Intelligence (AI) and Machine Learning (ML) applications rely heavily on *deep learning* frameworks that simplify the implementation and training of complex neural networks.

Two of the most widely used open-source libraries for this purpose are **TensorFlow** and **PyTorch**. Both are built for **numerical computation using dataflow graphs** and provide powerful APIs for building Artificial Neural Networks (ANNs) efficiently.

2.1 What is TensorFlow?

TensorFlow is an open-source platform for machine learning and deep learning developed by Google. It allows researchers and developers to build and train neural networks using multi-dimensional arrays called **tensors**.

Key Features of TensorFlow

- **Tensors:** Core data structure; represents data as n-dimensional arrays.
- **Computation Graphs:** Operations are represented as nodes in a graph (static or eager execution mode).
- **Keras API:** High-level interface in TensorFlow that simplifies model creation, training, and evaluation.
- **GPU/TPU Support:** TensorFlow can leverage hardware acceleration for faster training.
- **Model Deployment:** Integration with TensorFlow Lite, TensorFlow.js, and TensorFlow Serving allows models to be deployed on mobile, web, and production servers.

2.2 What is PyTorch?

PyTorch is an open-source deep learning framework developed by Facebook (Meta) that provides a flexible and dynamic way to define and train neural networks. Unlike TensorFlow's early versions, which used static computation graphs, PyTorch uses **dynamic computation graphs**, allowing changes in network structure during runtime, very useful for research and experimentation.

Key Features of PyTorch

- **Dynamic Computation Graphs:** Build and modify networks on the fly during execution.
- **Pythonic Syntax:** Seamless integration with Python libraries such as NumPy.
- **Automatic Differentiation:** The `autograd` module computes gradients automatically for backpropagation.
- **TorchScript & Deployment:** Models can be exported for production with TorchScript or deployed via ONNX.
- **GPU Support:** Built-in CUDA support for efficient GPU computations.

3 Single-Layer Perceptron using Pytorch

3.1 Import necessary libraries

If pytorch is not installed already: `pip install torch torchvision torchaudio`

torch: main PyTorch library

torch.nn: contains neural network layers

torch.optim: contains optimization algorithms

```
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

3.2 Create and split synthetic dataset

We will create a simple 2-feature synthetic binary-classification dataset for our demonstration and then split it into training and testing.

```
X, y = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_repeated=0,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
```

3.3 Standardize the Dataset

Now standardize the dataset to enable faster and more precise computations. Standardization helps the model converge more quickly and often enhances accuracy.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3.4 Convert Data to PyTorch Tensors

PyTorch works with a special data structure called a Tensor. A tensor is similar to a NumPy array, but it allows efficient computation and supports automatic differentiation for neural networks.

You can think of a tensor as a multi-dimensional array, similar to a NumPy array, but with additional capabilities.

Tensor type	Example	Dimensions
Scalar	5	0D
Vector	[1,2,3]	1D
Matrix	[[1,2],[3,4]]	2D
Higher-dim tensor	image data	3D+

```
import torch

a = torch.tensor(5)           # scalar
b = torch.tensor([1, 2, 3])   # vector
c = torch.tensor([[1, 2], [3, 4]]) # matrix
```

```
tensor(5)
tensor([1, 2, 3])
tensor([[1, 2],
        [3, 4]])
```

Common Tensor Data Types

The `dtype` determines how numbers are stored in memory.

dtype	Description
torch.int8	8-bit integer
torch.int16	16-bit integer
torch.int32	32-bit integer
torch.int64	64-bit integer (default for integers)
torch.float16	half precision
torch.float32	standard float (most common)
torch.float64	double precision

And so on...

```
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train,
dtype=torch.float32).reshape(-1,1)#reshaping to convert the target
vector into a column vector.

y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1,1)
```

```
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
● print(y_test.shape)
```

✓ 0.0s

```
torch.Size([800, 2])
torch.Size([800, 1])
torch.Size([200, 2])
torch.Size([200, 1])
```

3.5 Building a neural network

In PyTorch, neural networks are created by defining a class that inherits from **nn.Module**. `nn.Module` is the base class for all neural networks in PyTorch. It provides important functionality such as:

- registering layers
- storing model parameters
- enabling gradient computation
- allowing `.train()` and `.eval()` modes
- enabling `.parameters()` for optimizers

Common PyTorch Layers

Layer	PyTorch Syntax	Purpose	Typical Use
Fully Connected (Dense)	<code>nn.Linear(in_features, out_features)</code>	Connects every input neuron to every output neuron	Standard neural networks
1D Convolution	<code>nn.Conv1d(in_channels, out_channels, kernel_size)</code>	Convolution for sequential data	Time series, NLP
2D Convolution	<code>nn.Conv2d(in_channels, out_channels, kernel_size)</code>	Convolution for image data	Computer vision
3D Convolution	<code>nn.Conv3d(in_channels, out_channels, kernel_size)</code>	Convolution for volumetric data	Medical imaging, video
Max Pooling	<code>nn.MaxPool2d(kernel_size)</code>	Reduces spatial size by taking maximum value	CNN feature extraction

Average Pooling	<code>nn.AvgPool2d(kernel_size)</code>	Reduces spatial size by averaging values	CNN smoothing
Dropout	<code>nn.Dropout(p)</code>	Randomly disables neurons during training	Prevent overfitting
Batch Normalization	<code>nn.BatchNorm1d()</code> / <code>nn.BatchNorm2d()</code>	Normalizes layer outputs	Faster and stable training
Flatten	<code>nn.Flatten()</code>	Converts multidimensional input into vector	Before dense layers
Embedding	<code>nn.Embedding(num_embeddings, embedding_dim)</code>	Converts discrete tokens to dense vectors	NLP models
Recurrent Neural Network	<code>nn.RNN()</code>	Processes sequential data	Text, time series
Long Short-Term Memory	<code>nn.LSTM()</code>	Advanced RNN for long-term dependencies	NLP, speech
Gated Recurrent Unit	<code>nn.GRU()</code>	Simplified LSTM	Sequential models

Common Activation functions

Activation Function	PyTorch Syntax	Output Range	Typical Use
Sigmoid	<code>torch.sigmoid(x)</code>	$(0, 1)$	Binary classification
ReLU	<code>torch.relu(x)</code>	$[0, \infty)$	Most deep neural networks
Leaky ReLU	<code>nn.LeakyReLU()</code>	$(-\infty, \infty)$ small negative slope	Prevent dead neurons
Tanh	<code>torch.tanh(x)</code>	$(-1, 1)$	RNNs, older networks
Softmax	<code>torch.softmax(x, dim=1)</code>	Probabilities sum to 1	Multi-class classification
GELU	<code>nn.GELU()</code>	Smooth nonlinear activation	Transformers (BERT, GPT)
ELU	<code>nn.ELU()</code>	Smooth negative values	Deep networks
Softplus	<code>nn.Softplus()</code>	Smooth ReLU-like function	Numerical stability
Identity (No activation)	None	$(-\infty, \infty)$	Regression output

Typical Layer & Activation Combinations

Problem Type	Final Layer	Activation
Binary Classification	nn.Linear(...,1)	Sigmoid
Multi-Class Classification	nn.Linear(...,num_classes)	Softmax
Regression	nn.Linear(...,1)	None
Deep Neural Network Hidden Layer	nn.Linear(...)	ReLU

Our perceptron will contain:

- One linear layer
- One neuron
- Sigmoid activation function

The sigmoid function converts outputs into values between 0 and 1, which is useful for binary classification.

```
class Perceptron(nn.Module):

    def __init__(self):
        super(Perceptron, self).__init__() # calls parent class
        constructor
        self.linear = nn.Linear(2, 1)      # 2 input features, 1 output

    def forward(self, x):
        x = self.linear(x)
        x = torch.sigmoid(x)
        return x
```

```
model = Perceptron()
```

This creates an instance of the neural network. Internally PyTorch:

- Creates the Linear layer
- Initializes random weights
- Registers parameters for training

```
print(model)
for param in model.parameters():
    print(param)
```

```
Perceptron(
  (linear): Linear(in_features=2, out_features=1, bias=True)
)
Parameter containing:
tensor([[ -0.4016, -0.0630]], requires_grad=True)
Parameter containing:
tensor([ -0.1735], requires_grad=True)
```

The above class implementation is helpful when you need to control the forward pass. When you just need a sequence of layers you can use `nn.Sequential`

```
model=nn.Sequential(nn.Linear(2,1),nn.Sigmoid())
```

```
Sequential(
  (0): Linear(in_features=2, out_features=1, bias=True)
  (1): Sigmoid()
)
Parameter containing:
tensor([[ -0.4044,  0.2737]], requires_grad=True)
Parameter containing:
tensor([0.6142], requires_grad=True)
```

3.6 Define Loss Function and Optimizer

To train the neural network we need

1. Loss Function: Measures how different the predicted output is from the true output. For binary classification we use **Binary Cross Entropy Loss**

- Optimizer: Updates the model weights to reduce the loss. We are using the Adam optimizer, which is commonly used in deep learning.

Common Loss Functions in PyTorch

Loss Function	PyTorch Syntax	Used For	Description
Mean Squared Error	nn.MSELoss()	Regression	Measures average squared difference between predicted and true values
Mean Absolute Error	nn.L1Loss()	Regression	Measures average absolute difference between predictions and targets
Binary Cross Entropy	nn.BCELoss()	Binary classification	Compares predicted probabilities with true labels (0 or 1)
Binary Cross Entropy with Logits	nn.BCEWithLogitsLoss()	Binary classification	Combines sigmoid activation and BCE loss in a numerically stable way
Cross Entropy Loss	nn.CrossEntropyLoss()	Multi-class classification	Combines softmax activation and log loss
Negative Log Likelihood	nn.NLLLoss()	Classification	Used with log-softmax outputs
Hinge Loss	nn.HingeEmbeddingLoss()	Margin-based classification	Used in SVM-like models

Common Optimizers in PyTorch

Optimizer	PyTorch Syntax	Key Idea	Typical Use
Stochastic Gradient Descent	optim.SGD()	Updates weights using gradients	Basic optimization
Adam	optim.Adam()	Adaptive learning rate for each parameter	Most commonly used

RMSProp	<code>optim.RMSprop()</code>	Adapts learning rate using moving averages	RNN training
Adagrad	<code>optim.Adagrad()</code>	Adjusts learning rate based on past gradients	Sparse data
AdamW	<code>optim.AdamW()</code>	Adam with better weight decay handling	Modern deep learning models

```

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

```

3.7 Train the Model

```

epochs = 50

for epoch in range(epochs):
    # Forward pass
    predictions = model(X_train)
    # Compute loss
    loss = criterion(predictions, y_train)
    # Reset gradients
    optimizer.zero_grad()
    # Backpropagation
    loss.backward()
    # Update weights
    optimizer.step()
    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

```

This code performs the training process of the neural network by repeatedly updating the model's weights so that its predictions become closer to the true labels. The line `optimizer.zero_grad()` clears any gradients stored from the previous iteration, because PyTorch accumulates gradients by default. Then `loss.backward()` performs backpropagation, where PyTorch automatically computes the gradients of the loss with respect to all learnable parameters in the model using the chain rule of calculus. These gradients indicate how the weights should change to reduce the loss. After that, `optimizer.step()` updates the model's weights using the chosen optimization

algorithm (Adam in this case), moving them in the direction that reduces the loss. This sequence of (1) forward pass (2) loss computation (3) gradient calculation (4) weight update repeats for every epoch, gradually improving the model's predictions.

```
Epoch 10, Loss: 0.6931
Epoch 20, Loss: 0.6535
Epoch 30, Loss: 0.6210
Epoch 40, Loss: 0.5934
Epoch 50, Loss: 0.5694
```

3.8 Model Evaluation

After training we test the model's performance on unseen data.

```
with torch.no_grad(): #Do not track gradients for the operations inside
    this block.
    y_pred_prob = model(X_test)
    y_pred = (y_pred_prob > 0.5).float()
    accuracy = (y_pred == y_test).sum().item() / y_test.size(0)
print("Accuracy:", accuracy)
```

Accuracy: 0.88

If you want more detailed metrics such as confusion matrix, F1-score, or classification report use scikit-learn

```
from sklearn.metrics import confusion_matrix, classification_report

y_pred_np = y_pred.numpy()
y_test_np = y_test.numpy()

print(confusion_matrix(y_test_np, y_pred_np))
print(classification_report(y_test_np, y_pred_np))
```

```
[[89 12]  
 [12 87]]
```

	precision	recall	f1-score	support
0.0	0.88	0.88	0.88	101
1.0	0.88	0.88	0.88	99
accuracy			0.88	200
macro avg	0.88	0.88	0.88	200
weighted avg	0.88	0.88	0.88	200